

Trabalhando com Arquivos em Java

Parte 2 — Leitura e Escrita de Dados em Arquivos

Do fluxo clássico de bytes e caracteres do java.io até os métodos modernos da classe Files, passando por conjuntos de caracteres, encodings e as classes de escrita BufferedWriter, FileWriter e PrintWriter. Exemplos aplicados a um sistema de pedidos de uma pizzeria.

CONTEÚDO DA APOSTILA

- 01** Lendo Texto: o Jeito Antigo (java.io)
- 02** Conjuntos de Caracteres e Encodings
- 03** Lendo Texto de um Arquivo com NIO.2
- 04** Escrevendo Dados em um Arquivo
- 05** BufferedWriter, FileWriter e PrintWriter
- 06** Flush: Entendendo o Buffer de Escrita

Sumário

0 1 Lendo Texto: o Jeito Antigo (java.io)

InputStream, Reader, BufferedReader e o custo de uma leitura em disco

0 2 Conjuntos de Caracteres e Encodings

ASCII, Unicode, UTF-8 e como o Java representa texto internamente

0 3 Lendo Texto de um Arquivo com NIO.2

Os métodos modernos de leitura da classe Files, em uma única chamada

0 4 Escrevendo Dados em um Arquivo

OutputStream, FileWriter e as opções padrão de abertura de arquivo

0 5 BufferedWriter, FileWriter e PrintWriter

Comparando as três classes de escrita de texto mais usadas

0 6 Flush: Entendendo o Buffer de Escrita

Quando os dados realmente chegam ao disco, e como controlar isso

Lendo Texto: o Jeito Antigo (java.io)

Entendendo o fluxo de bytes e caracteres por trás de `Files.readAllLines`

A forma mais simples de ler um arquivo de texto em Java hoje é uma única chamada a **`Files.readAllLines`**, e essa é a recomendação para a grande maioria dos casos. Mas para entender como esse método mágico funciona por dentro, vale voltar no tempo até as classes originais do java.io, disponíveis desde o Java 1.0 — elas refletem melhor o processo real de leitura de dados de um arquivo, ao contrário de um método caixa-preta como `readAllLines`.

O custo de uma leitura em disco

Uma **leitura em disco** significa que algo está acontecendo física ou mecanicamente no seu disco para trazer aquele caractere do arquivo. Essa operação é cara, tanto em tempo quanto em uso de recursos. Felizmente, o Java oferece formas de reduzir o número de leituras diretas ao disco — e é aí que entra o conceito de **buffer de arquivo**: memória usada temporariamente para guardar dados enquanto são lidos do arquivo. Seu propósito principal é melhorar a eficiência da transferência e do processamento de dados, reduzindo as interações diretas com o dispositivo de armazenamento.

InputStream: a fonte de bytes

InputStream é uma classe abstrata que representa um fluxo de entrada de bytes — uma fonte de dados e uma interface comum para lê-los. Para arquivos, a implementação concreta é a **`FileInputStream`**, usada para arquivos com dados binários (voltaremos a ela na próxima apostila, sobre `RandomAccessFile` e serialização). Usar o método `read` diretamente numa `FileInputStream` é muito ineficiente — se você precisar de uma `FileInputStream`, o recomendado é envolvê-la em uma **`BufferedInputStream`**.

NOTA

Um `InputStream` **não** é a mesma coisa que um `Stream` de pipeline (aquele usado com `filter`, `map` e `collect`). É um conceito parecido — dados chegando de forma sequencial — mas um `InputStream` não pode ser usado diretamente num pipeline de `Stream` sem antes ser transformado.

Readers: lendo caracteres, não bytes

Enquanto os `InputStreams` trabalham com bytes, os **Readers** trabalham com caracteres. Um **`InputStreamReader`** funciona como uma ponte entre fluxos de bytes e fluxos de caracteres. Para ler um fluxo de caracteres diretamente, o recomendado é usar **`FileReader`**, que já faz leitura bufferizada por padrão. Já o **`BufferedReader`** também faz leitura bufferizada, mas usando um buffer bem maior que o da `FileReader` — esse tamanho pode ser configurado manualmente, embora o valor padrão já seja adequado para a grande maioria dos casos. O `BufferedReader` ainda oferece métodos de conveniência para leitura linha a linha, como o clássico `readLine`.

```
// Lendo o cardápio da pizzaria linha a linha, no estilo java.io
Path cardapio = Paths.get("pizzaria", "cardapio.txt");
try (BufferedReader leitor = new BufferedReader(
    new FileReader(cardapio.toFile()))) {
    String linha;
    while ((linha = leitor.readLine()) != null) {
        System.out.println(linha);
    }
} catch (IOException e) {
    System.out.println("Erro ao ler o cardápio: " + e.getMessage());
}
```

Conjuntos de Caracteres e Encodings

ASCII, Unicode e como o Java decide que bytes representam qual texto

O que é um conjunto de caracteres

Um **conjunto de caracteres** (character set) é uma coleção definida de símbolos, letras, números, sinais de pontuação e outros caracteres. Cada caractere do conjunto recebe um código numérico único, chamado **code point**, que permite ao computador armazenar, transmitir e interpretar texto. Os dois conjuntos mais comuns são o ASCII — o mais antigo e amplamente usado — e o Unicode, mais recente, criado para suportar todos os sistemas de escrita do mundo.

A **codificação de caracteres** (character encoding) é o processo de atribuir números aos diferentes caracteres, chamados **glyphs** — que podem ser letras de qualquer idioma, pontuação ou até emojis. Existem diferentes formas de representar um glyph através de um valor numérico, e é exatamente isso que diferencia os encodings entre si.

Encodings ASCII

Encoding	Tamanho	Inclui alfabeto latino?	Observações
US-ASCII	7 bits	Não	Faixa menor de caracteres suportados
ISO-8859-1	8 bits	Sim	Mais amplamente suportado que o US-ASCII

Encodings Unicode

Encoding	Tamanho	Benefícios
UTF-8	Variável (1 a 4 bytes)	O encoding mais popular na internet. Inclui o ISO-8859-1 e muito mais. Consegue representar caracteres de todos os sistemas de escrita.
UTF-16	2 bytes	Amplamente suportado
UTF-32	4 bytes	Mais eficiente e direto de processar, mas ocupa mais espaço de armazenamento. Raramente usado.

Qual encoding usar?

Na maioria das aplicações, **UTF-8** é a melhor escolha: é eficiente, amplamente suportado, e consegue representar uma faixa muito mais ampla de caracteres. No entanto, se sua aplicação trabalha exclusivamente com caracteres ASCII, o ISO-8859-1 pode ser uma escolha mais eficiente. O Java concentra os encodings mais comuns num enum chamado **StandardCharsets**, evitando erros de digitação em Strings soltas como "UTF-8".

```
// Lendo um pedido com o encoding explícito
Path pedido = Paths.get("pizzaria", "pedido-102.txt");
String conteudo = Files.readString(pedido, StandardCharsets.UTF_8);
System.out.println(conteudo);
```

Lendo Texto de um Arquivo com NIO.2

Os métodos modernos da classe `Files`, para o dia a dia

Depois de entender o processo manual com `Readers` e `InputStreams`, fica mais fácil apreciar a conveniência dos métodos modernos de leitura oferecidos pela classe **`Files`**. Todos eles leem o conteúdo inteiro do arquivo para a memória de uma só vez, e por isso são recomendados para arquivos de até aproximadamente 2 gigabytes.

Assinatura do método	Descrição	Fecha o arquivo?
<code>byte[] readAllBytes(Path path)</code>	Lê todo o conteúdo de qualquer arquivo em um array de bytes	Sim
<code>String readString(Path path)</code>	Lê todo o conteúdo de um arquivo de texto em uma <code>String</code>	Sim
<code>List<String> readAllLines(Path path)</code>	Lê todo o conteúdo de um arquivo de texto em uma lista de <code>Strings</code>	Sim
<code>Stream<String> lines(Path path)</code>	Lê todo o conteúdo de um arquivo de texto como um <code>Stream</code>	Na operação terminal

Escolhendo o método certo

Para a maioria dos casos — um arquivo de configuração, um pedido, um relatório pequeno — **`readString`** ou **`readAllLines`** são as escolhas mais diretas. Já o método **`lines`** retorna um `Stream` preguiçoso (lazy), o que o torna a melhor opção quando você quer processar um arquivo grande linha a linha sem carregar tudo de uma vez na memória — mas, por só fechar o arquivo na operação terminal, ele deve sempre ser usado dentro de um `try-with-resources`.

```
// Contando quantos pedidos de calabresa existem no dia, com NIO.2
Path pedidosDoDia = Paths.get("pizzaria", "pedidos-2026-07-14.txt");
// Carregando tudo de uma vez, ideal para arquivos pequenos:
List<String> linhas = Files.readAllLines(pedidosDoDia, StandardCharsets.UTF_8);
long calabresas = linhas.stream()
    .filter(l -> l.contains("Calabresa"))
    .count();
// Processando como Stream, ideal para arquivos grandes:
try (Stream<String> linhasStream = Files.lines(pedidosDoDia)) {
    long total = linhasStream
        .filter(l -> l.contains("Calabresa"))
        .count();
    System.out.println("Pedidos de calabresa: " + total);
}
```

DICA

`Files.readString` existe desde o Java 11 e é a forma mais direta de trazer o conteúdo inteiro de um arquivo de texto pequeno para uma única String — sem precisar concatenar linha por linha manualmente.

Escrevendo Dados em um Arquivo

Do OutputStream ao FileWriter, e as opções padrão de abertura

Por que escrever em arquivos

Existem muitos motivos para uma aplicação escrever dados em arquivo: guardar dados de usuário, registrar eventos da aplicação em um arquivo de log, armazenar dados de configuração, exportar dados para troca de informações, dar suporte a uso offline através de um cache em arquivo, ou gerar produtos de arquivo (como um relatório em PDF ou CSV).

Escrever é tão diferente de ler?

Alguns conceitos de escrita em arquivo são naturalmente parecidos com os de leitura. As classes usadas têm nomes similares, mas espelhados: no lugar de `InputStream`, trabalha-se com **`OutputStream`**; no lugar de `FileReader`, existe a classe **`FileWriter`**. Entender o buffer de dados se torna ainda mais importante na escrita, assim como gerenciar múltiplas escritas concorrentes vindas de diferentes threads para o mesmo arquivo. Além disso, existem diferentes formas de abrir um arquivo para escrita.

Opções padrão de abertura

Todas as opções disponíveis estão em um enum do pacote `java.nio.file` chamado **`StandardOpenOption`**. As opções padrão usadas pelos métodos `Files.write` são:

Opção	Descrição
CREATE	Cria um novo arquivo, caso ele ainda não exista
TRUNCATE_EXISTING	Se o arquivo já existir e for aberto para escrita, seu conteúdo é truncado para 0
WRITE	O arquivo é aberto para acesso de escrita

```
// Gravando um novo pedido no arquivo do dia
Path pedidosDoDia = Paths.get("pizzaria", "pedidos-2026-07-14.txt");
String novoPedido = "Pedido #103 - Pizza Calabresa - Bairro Centro";
// Sobrescreve o conteúdo (comportamento padrão: CREATE + TRUNCATE_EXISTING + WRITE)
Files.writeString(pedidosDoDia, novoPedido);
// Anexando ao final do arquivo, sem apagar o conteúdo anterior:
Files.writeString(pedidosDoDia, System.lineSeparator() + novoPedido,
StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

ATENÇÃO

Sem a opção APPEND, cada chamada a `Files.writeString` ou `Files.write` sobrescreve completamente o conteúdo anterior do arquivo, por causa da opção padrão `TRUNCATE_EXISTING`. Para adicionar conteúdo ao final, é obrigatório informar APPEND explicitamente.

BufferedWriter, FileWriter e PrintWriter

Comparando as três classes de escrita de texto mais usadas

Além dos métodos utilitários de Files, o Java oferece três classes de escrita de texto com propósitos ligeiramente diferentes. Conhecer as diferenças ajuda a escolher a ferramenta certa para cada situação.

Classe	Buffer	Formato de dados	Uso recomendado
BufferedWriter	Sim	Caracteres	Escrever grandes volumes de texto em um arquivo; possui o método <code>newLine</code> para quebras de linha
FileWriter	Sim, mas bem menor que o do <code>BufferedWriter</code>	Caracteres	Escrever pequenos volumes de texto; não tem método próprio para separador de linha — seria preciso escrever manualmente
PrintWriter	Não tem buffer próprio, mas costuma envolver um <code>BufferedWriter</code>	Caracteres, números e objetos	Escrever texto formatado em arquivo, com métodos familiares — o mesmo comportamento de <code>System.out</code>

```
// Gerando um relatório de pedidos do dia com PrintWriter + BufferedWriter
Path relatorio = Paths.get("pizzaria", "relatorio-2026-07-14.txt");
try (PrintWriter escritor = new PrintWriter(
    new BufferedWriter(new FileWriter(relatorio.toFile())))) {
    escritor.println("Relatório de Pedidos - 14/07/2026");
    escritor.printf("Pedido #%d - %s - R$ %.2f%n", 101, "Marguerita", 42.90);
    escritor.printf("Pedido #%d - %s - R$ %.2f%n", 102, "Calabresa", 39.90);
} catch (IOException e) {
    System.out.println("Erro ao gerar relatório: " + e.getMessage());
}
```

DICA

`PrintWriter` é a escolha mais natural quando você já está acostumado com `System.out.println` e `System.out.printf` — o comportamento é praticamente idêntico, só que a saída vai para um arquivo em vez do console. Diferente de outros Writers, seus métodos não lançam `IOException` diretamente; em caso de erro, é possível verificar com `checkError()`.

Estruturando texto com StringJoiner

Quando o objetivo é montar um texto com um delimitador fixo entre elementos — por exemplo, um formato tipo CSV ou uma lista separada por vírgulas dentro de colchetes — a classe **`StringJoiner`** é uma alternativa mais legível do que concatenar Strings manualmente, pois já permite definir prefixo, sufixo e delimitador de uma vez.

```
// Montando uma linha de pedido separada por vírgulas com StringJoiner
StringJoiner linha = new StringJoiner(", ", "[", "]");
linha.add("Pedido #103");
linha.add("Calabresa");
linha.add("R$ 39,90");
System.out.println(linha);
// Saída: [Pedido #103, Calabresa, R$ 39,90]
```

Flush: Entendendo o Buffer de Escrita

Quando os dados realmente chegam ao disco

Ao escrever em um arquivo, existe um armazenamento temporário — o buffer — que vai se enchendo à medida que operações de escrita são executadas em uma classe `Writer`. A escrita física em disco só acontece quando esse buffer é **flushado** (flush): o processo de pegar o texto armazenado no buffer, escrevê-lo de fato no arquivo de saída, e limpar o cache do buffer.

A frequência com que o flush ocorre pode ser afetada por vários fatores, incluindo o tamanho do buffer, a velocidade do disco, e o volume de dados sendo escrito no arquivo.

Quando o flush acontece

O buffer é **sempre** flushado quando o arquivo é fechado — por isso fechar corretamente um `Writer` (idealmente com `try-with-resources`) é tão importante quanto abri-lo. Também é possível forçar um flush manualmente, chamando o método **`flush()`**. Isso costuma ser útil quando se trabalha com dados sensíveis ao tempo, como um log que outro processo está monitorando em tempo real: até que o flush ocorra, nenhuma outra thread ou processo conseguirá enxergar o texto ainda armazenado apenas no buffer.

```
// Forçando o flush para que a cozinha veja o pedido imediatamente
Path filaCozinha = Paths.get("pizzaria", "fila-cozinha.log");
try (BufferedWriter escritor = Files.newBufferedWriter(
    filaCozinha, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
    escritor.write("Pedido #104 recebido às 20:14");
    escritor.newLine();
    escritor.flush(); // garante visibilidade imediata para o painel da cozinha
} catch (IOException e) {
    System.out.println("Erro ao registrar pedido na fila: " + e.getMessage());
}
```

ATENÇÃO

Fazer flush manual com muita frequência anula parte do ganho de desempenho do buffer, já que cada flush pode disparar uma escrita física em disco. Use-o deliberadamente — quando a visibilidade imediata dos dados realmente importa — e deixe o resto para o flush automático no fechamento do recurso.

Fim de apostila

Este capítulo cobriu o ciclo completo de leitura e escrita de texto em Java: das classes originais do `java.io` até os métodos modernos de `Files`, passando por conjuntos de caracteres, encodings e o papel do buffer nas classes de escrita. A próxima apostila avança para o acesso binário e aleatório a arquivos, com

RandomAccessFile e serialização de objetos.

JavaStudiesHub